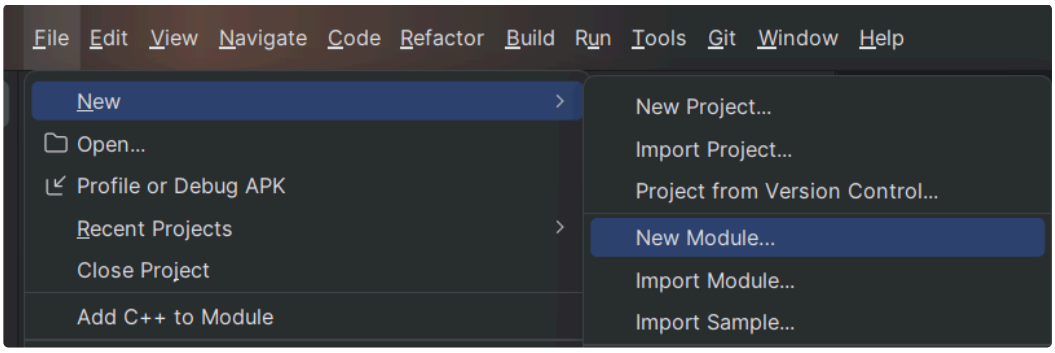
Modular feature - Maps

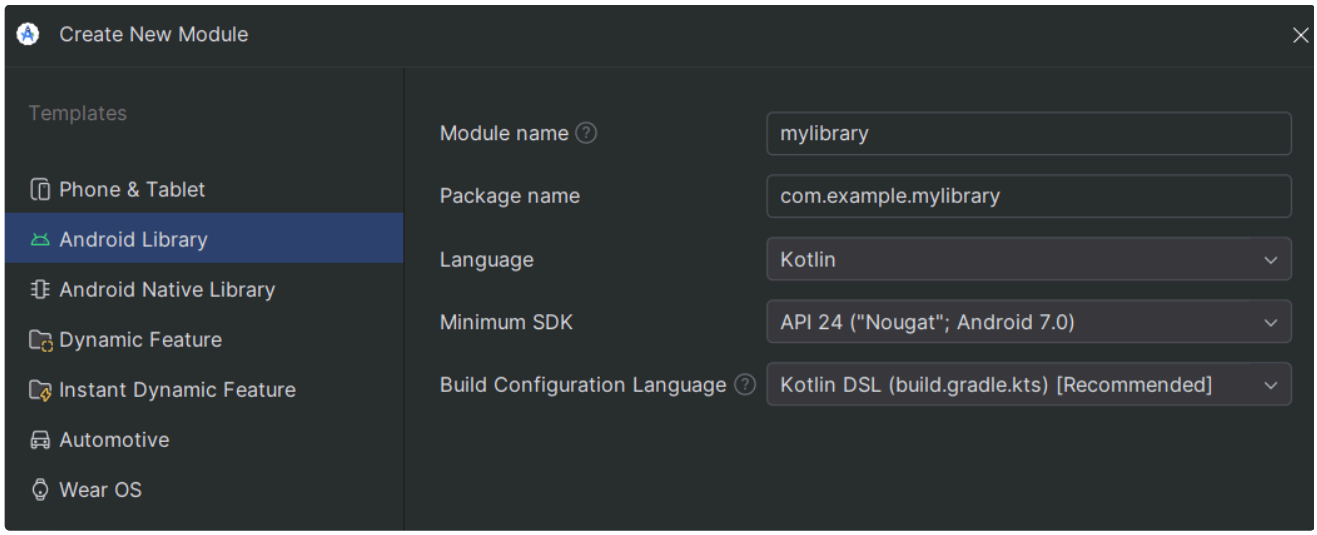
Welcome! Our ServiceSync app has a plugin functionality that is open to the community - maps. As of writing this page, ServiceSync has 2 implemented map providers - Google Maps and OSM. If you want to contribute to our app in a meaningful way, you can implement a new map provider without changing ServiceSync.

Contributing to the app by adding a map provider

Below are the steps describing how you can contribute to ServiceSync by adding a new map provider for our users.

1. Create a new module





2. Setup build.gradle.kts (for the new module)

Open the build.gradle.kts file for the new module

/ **Modular feature - Maps** 1 Jira link

```
api(project(":maps"))
```

Now change compileSDK to 35:

```
compileSdk = 35
```

Now add compose compiling:

```
buildFeatures {
        compose = true
    }
```

Lastly, add compose plugins:

```
plugins {
    alias(libs.plugins.android.library)
    alias(libs.plugins.kotlin.android)
    alias(libs.plugins.kotlin.compose)
}
```

3. Implement interface IMapProvider located in “maps” module

In the previous step, you imported our module “maps”. That module has the following interface:

```
interface IMapProvider {

    fun getName() : String

    @Composable
    fun CreateMap(latitude: Double, longitude: Double)

}
```

This is the interface that is needed so that ServiceSync recognizes a new map provider. Function explanation:

- **getName() : String** - like the name suggests, this method should be one line of code returning the name of the map provider (e.g. “Google Maps” or “Open Street Map”)
- **CreateMap(latitude: Double, longitude: Double)** - this is a composable function (it should return some kind of UI) where you receive longitude and latitude of the location and use these parameters to draw a map. Don’t worry about constraining the map to a certain height and width, ServiceSync does that for you automatically when the map is drawn on company location. It fits the map into a Box of 200 dp height and max width.

It doesn't matter if you implement this in one layer or multiple layers. By all means, you can create more layers in your module, but the UI layer class must implement IMapProvider interface.

/ **Modular feature - Maps** 1 Jira link

Android Studio doesn't do this automatically, you need to do it.

Go into your module. Create a directory called **resources (not res)**.

Android Studio will give you resource sample set and click on it.

After creating the resources directory, create a new directory called **META-INF** in the same way you created the resources directory. After that, create another directory called **services**.

Now, create a regular file called **com.example.maps.interfaces.IMapProvider**.

Open the file and in the first line, enter the class name that implements the interface **IMapProvider** with the package prefix.

```
com.example.google_maps.ui.GoogleMapsProvider
```

5. Add your module as a dependency to the app module in build.gradle.kts

/ **Modular feature - Maps** 1 Jira link

Add your dependency below google_maps and osm in the same way (for your module):

Now sync the app with the new gradle build.

6. Reviewing your changes

Run ServiceSync and open the profile screen.

/ **Modular feature - Maps** 1 Jira link

Your map should be in the dropdown. Select it, go to either **Favorites** or **Search** and find any company and open its details. Verify that your map doesn't break the boundaries.

Congratulations! Your maps have been added into ServiceSync! Create a PR so that our team can review your changes and add them into the app!

WARNING: If your map breaks the boundary on company location, your changes will be denied. You are only allowed to change the build.gradle.kts for the app module (adding the

/ **Modular feature - Maps** 1 Jira link

Technical details (how we implemented modular maps in ServiceSync)

For the tech savvy and those wondering how our team implemented this, this section is for you! By now, you know how to add your module and how IMapProvider interface works and what it requires. This section will focues entirely on the app module and how implementations of IMapProvider interface are found during runtime.

UI layer

For starters, let's open ProfileContent composable function.

```
ProfileInfoBox(stringResource(R.string.map_type), mapProvider)
```

This line of code adds a new infor box to the profile screen before editing. ProfileInfoBox is our own component (credit to @Ivan Beusan for making it) that just styles the text so it can be displayed.

mapProvider variable is a string retrieved in LaunchedEffect.

```
mapProvider = MapProviderManager.getCurrentMapProviderName(context)
```

```
MapDropdown(mapProviderName = mapProvider,onMapProviderChange = { newName ->
                        mapProvider = newName
                    }
                )
```

MapDropdown is a composable developed by our team, it requires the currently saved map provider name and a lambda function that gets called after a new map provider is saved. After the save button is clicked, map provider is saved by the business logic layer:

```
MapProviderManager.saveMapProvider(
                            context = context,
                            mapProviderName = mapProvider
                        )
```

MapDropdown:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MapDropdown(mapProviderName: String,onMapProviderChange : (String) -> Unit) {
    val  mapProviders = MapProviderManager.getAllProviders()
    var expanded by remember { mutableStateOf(false) }
```

```
    val dropdownTextFieldColors : TextFieldColors = if (mapProviders.isNotEmpty()) {
```

/ **Modular feature - Maps** 1 Jira link

```

        )
    }
    else {
        TextFieldDefaults.colors()
    }

    ExposedDropdownMenuBox(
        expanded = expanded,
        onExpandedChange = {expanded = !expanded},
        modifier = Modifier
    ){
        var chosenMapProvider by remember { mutableStateOf(mapProviderName)
         }
        TextField(
            label = {
                Text(text = stringResource(R.string.choose_map_type)) },
            value = chosenMapProvider,
            onValueChange = {
                chosenMapProvider = it
            },
            readOnly = true,
            enabled = false,
            colors = dropdownTextFieldColors,
            trailingIcon = {
                ExposedDropdownMenuDefaults.TrailingIcon(expanded = expanded)
            },
            modifier = Modifier.fillMaxWidth().menuAnchor(MenuAnchorType.PrimaryNotEd
        )
        ExposedDropdownMenu(expanded = expanded, onDismissRequest = {expanded = false
            mapProviders.forEach {
                DropdownMenuItem(
                    text = {
                        Text(text = it.getName())
                    },
                    onClick = {
                        expanded = false
                        chosenMapProvider = it.getName()
                        onMapProviderChange(chosenMapProvider)
                    }
                )
            }
        }
    }
}
```

At the very beggining, it loads all map providers. If no map providers exist, it sets the colors of

/ **Modular feature - Maps** 1 Jira link

visually disabled and we don't want that if there are map providers) ExposedDropDownMenuBox requires a TextField and ExposedDropdownMenu in the content (see docs). For every implementation of IMapProvider interface, a new DropdownListItem is created. On click, it closes the dropdown and updates the current map provider name.

Now let's look at how *CreateMap* method is called on company details, specifically *CompanyDetailsContent*.

```
if (MapProviderManager.getAllProviders().isNotEmpty()) {
    ListItem(
        headlineContent = {
            Text(
                text = stringResource(id = R.string.location),
                color = isDark(onSurfaceDark, onSurfaceLight),
                style = headlineTextStyle,
                modifier = Modifier.fillMaxWidth().padding(vertical = 10.dp)
            )
        },
        supportingContent = {
            companyGeoPoint.value?.let { geoPoint ->
                Log.d("Firestore", "Supporting content geopoint: $geoPoint")
                CompanyLocation(geoPoint = geoPoint)
            } ?: Text("Location data is unavailable.")
        }

    )
}
```

Company details is a big list of headings and content. If no map providers exist, the "Location" header and the box showing the map are not even shown. If there are any, it chooses to display the list item.

```
@Composable
fun CompanyLocation(geoPoint: GeoPoint)
{
    val mapProvider = MapProviderManager.getCurrentMapProvider(context = LocalContext
    if (mapProvider != null) {
        Box(modifier = Modifier
            .fillMaxWidth()
            .height(200.dp)
            .clipToBounds()
        ) {
            mapProvider.CreateMap(geoPoint.latitude,geoPoint.longitude)
        }
    }
```

```
    else {
        Text(text = stringResource(R.string.map_not_available))
```

/ **Modular feature - Maps** 1 Jira link

As a extra layer of security, company locations has a check if the retrieved map provider is *null*. If it is not, it displays a box with 200 dp height and full width and calls the composable function *CreateMap*. If `getCurrentMapProvider` returns null, the composable displays a static string informing the user that the map cannot be drawn at the moment.

Business logic layer

MapProviderManager is a singleton object on the business logic layer of the app that handles loading all implementations of the IMapProvider interface. This singleton has its own interface that it implements, if in the future current implementations will not work. Let's look at the interface **IMapProviderStateManager.**

```
interface IMapProviderStateManager {
    fun getAllProviders() : List<IMapProvider>

    fun saveMapProvider(context: Context, mapProviderName: String)

    fun getCurrentMapProviderName(context: Context) : String

    fun getCurrentMapProvider(context: Context) : IMapProvider?
}
```

Any class or singleton that implements this interface should be on the business logic layer and should implements the following methods:

- **getAllProviders -** this method must return a list of all map providers during runtime. **If no providers exist at the moment, this method must return a empty list, not null.** This method is used to populate a dropdown menu
- **saveMapProvider** - this method should contact the data access layer of the app for storing a new map provider. This method is called when user saves changes on the profile screen.
- **getCurrentMapProviderName -** this method should return a string of the currently saved map provider. This method is called when the user opens the profile screen and the app needs to display the currently chosen map provider name in the info box and the initially selected item in the dropdown menu
- **getCurrentMapProvider** - this method is used on CompanyLocation composable to get the object of the class that implements the IMapProvider interface so that the composable function *CreateMap* can be called and the map can be drawn

Here is how *MapProviderManager* implements this interface:

```
object MapProviderManager : IMapProviderStateManager {
```

```
    private const val KEY = "MAP_PROVIDER"
    private val mapProviders by lazy {
```

/ **Modular feature - Maps** 1 Jira link

```
    private fun getFirstMapProviderOrErrorMessage(context: Context) : String {
        return mapProviders.firstOrNull()?.getName() ?: context.getString(R.string.no
    }

    override fun getAllProviders() : List<IMapProvider> {
        return mapProviders
    }

    override fun saveMapProvider(context: Context, mapProviderName: String) {
        val mapProviderExists = mapProviders.any { mp -> mp.getName() == mapProviderN
        if (mapProviderExists) {
            SharedPreferencesManager.saveStringPreference(context,KEY,mapProviderName
        }
        else {
            Log.e("NONEXISTANT_MAP_PROVIDER", "Korisnik je pokušao spremiti karte koj
        }
    }

    override fun getCurrentMapProviderName(context: Context) : String {
        val storedMapProvider = SharedPreferencesManager.getMapProvider(context, KEY)
        val foundProvider = mapProviders.any { provider -> provider.getName() == stor
        return if (foundProvider) {
            storedMapProvider
        } else {
            getFirstMapProviderOrErrorMessage(context)
        }
    }

    override fun getCurrentMapProvider(context: Context): IMapProvider? {
        return mapProviders.find {mapProvider -> mapProvider.getName() == getCurrentM
    }

}
```

ServiceLoader is a java class that is used to locate all implementations of a interface. This class requires **META-INF/services** directory in order to discover implementations during runtime. Implementations are loaded on method *load* where the interface is specified and is transformed to a list. If at runtime no implementations exist, the list is allocated but it is empty (size = 0).

- **getAllProviders** - simply returns the private list *mapProviders*
- **saveMapProvider -** uses a custom singleton *SharedPreferencesManager* to store the ma provider name. However, it verifies that there is a map provider with that name in the list before saving. If not, it doesn't save (this is needed when no providers exist, then a static string will be sent to this method).

- **getCurrentMapProviderName -** like saving, it uses *SharedPreferencesManager* to retrieve the name of the provider. If shared preferences doesn't have a map provider saved, it will return

  / **Modular feature - Maps** 1 Jira link

  and if it does, returns the first loaded map provider. If no map providers exist, returns a static string informing the user that no providers exist.
- **getCurrentMapProvider -** uses kotlin method *find* to return *null* or a object implementing *IMapProviderInterface*

Data access layer

Like the business logic layer, the data access layer also has a custom interface should we one day decide to store map providers somewhere besides shared preferences.

```
interface IChosenMapProviderSaver {
    fun saveMapProvider(context: Context,key: String, value: String)

    fun getMapProvider(context: Context, key: String) : String?
}
```

- **saveMapProvider** - this method must somehow store the name of the chosen map provider
- **getMapProvider -** this method must retrieve the name of the map provider or *null* if not found

The singleton that implements this interface is **SharedPreferencesManager.**

```
object SharedPreferencesManager : IChosenMapProviderSaver {

    fun saveStringPreference(context: Context, key: String, value: String) {
        val sharedPreferences = PreferenceManager.getDefaultSharedPreferences(context

        with(sharedPreferences.edit()) {
            putString(key,value)
            apply()
        }
    }

    fun getStringPreference(context: Context,key: String) : String? {
        val sharedPreferences = PreferenceManager.getDefaultSharedPreferences(context

        return sharedPreferences.getString(key,null)
    }

    override fun saveMapProvider(context: Context, key: String, value: String) {
        saveStringPreference(context,key,value)
    }

    override fun getMapProvider(context: Context,key: String) : String? {
        return getStringPreference(context,key)
```

```
    }
}
```

/ **Modular feature - Maps** 1 Jira link

getStringPreference. It is implemented this way to allow other parts of the app usage of the singleton.

- **saveMapProvider -** gets the shared preferences and stores a string into them using the key
- **getMapProvider** - gets the shared preferences and retrieves a string by the provided key

Dependecy graph

UI sketches
ServiceSync

Notifications - Product requirements
ServiceSync

/ **Modular feature - Maps** 1 Jira link

Be the first to add a reaction